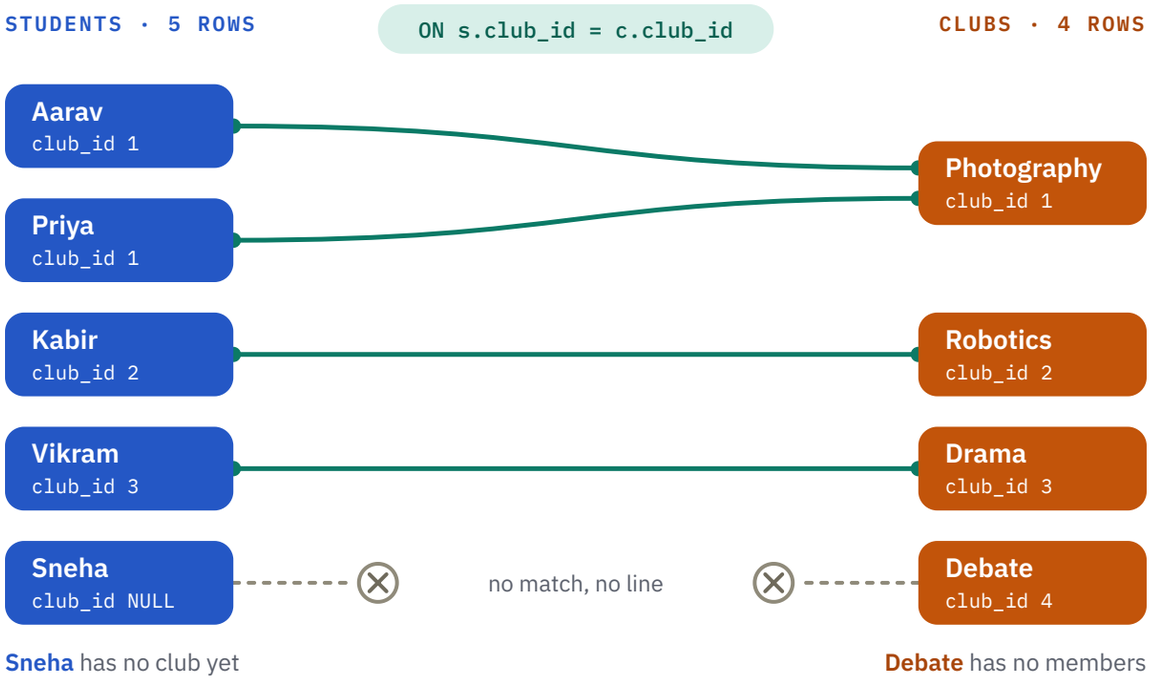
Two lists, one question.

Every join answers the same thing: what happens to the orphans? Here that is **Sneha** (no club) and **Debate** (no members).

THE CAST · A LINE = SAME club_id



Sneha has no club yet

Debate has no members

WHAT EACH JOIN KEEPS

	Matches in both	Sneha no club	Debate no members	Rows result
INNER	✓	✗	✗	4
LEFT	✓	✓	✗	5
RIGHT	✓	✗	✓	5
FULL	✓	✓	✓	6

Matches always survive. The keyword only decides which orphans stay.

THE REST OF THE FAMILY

CROSS JOIN
No ON. Every student × every club = 20 rows.
BOARD 3

SELF JOIN
One table joined to itself through two aliases (mentors).
BOARD 5

NATURAL / USING
Shorthand for matching on same-named columns.
BOARD 5

EXISTS / NOT EXISTS
A yes/no match check that never duplicates rows.
BOARD 6

NOT SURE WHICH?
Follow the flowchart, then skim the gotchas.
BOARD 7

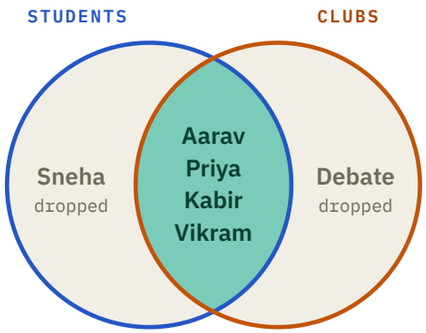
Same circles, different keeps.

Matched pair · Dropped · Sneha: student, no club · Debate: club, no members

INNER JOIN

Only real matches · printing badges

4 ROWS



STUDENT	CLUB
Aarav	Photography
Priya	Photography
Kabir	Robotics
Vikram	Drama

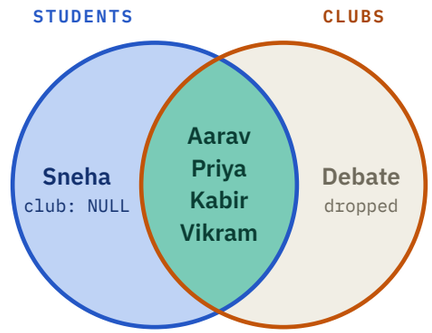
```
FROM students s INNER JOIN clubs c ON s.club_id = c.club_id
```

A bare **JOIN** means INNER JOIN.

LEFT JOIN

Every student · class roster

5 ROWS



STUDENT	CLUB
Aarav	Photography
Priya	Photography
Kabir	Robotics
Sneha	NULL
Vikram	Drama

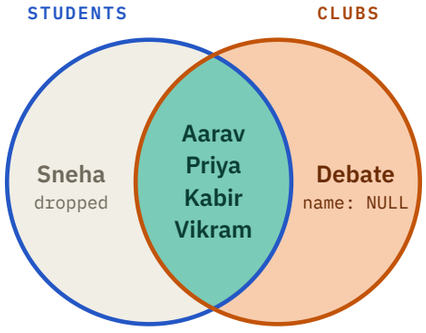
```
FROM students s LEFT JOIN clubs c ON s.club_id = c.club_id
```

LEFT = the table written *before* the word JOIN.

RIGHT JOIN

Every club · funding report

5 ROWS



STUDENT	CLUB
Aarav	Photography
Priya	Photography
Kabir	Robotics
Vikram	Drama
NULL	Debate

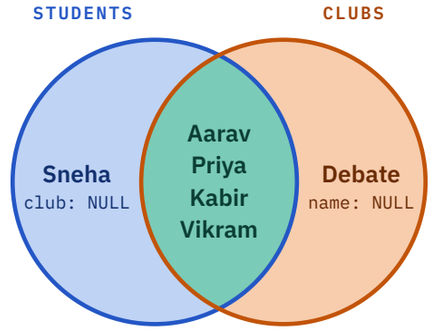
```
FROM students s RIGHT JOIN clubs c ON s.club_id = c.club_id
```

Rarely written: swap the two tables and use LEFT JOIN.

FULL JOIN

Everything from both · activity audit

6 ROWS



STUDENT	CLUB
Aarav	Photography
Priya	Photography
Kabir	Robotics
Sneha	NULL
Vikram	Drama
NULL	Debate

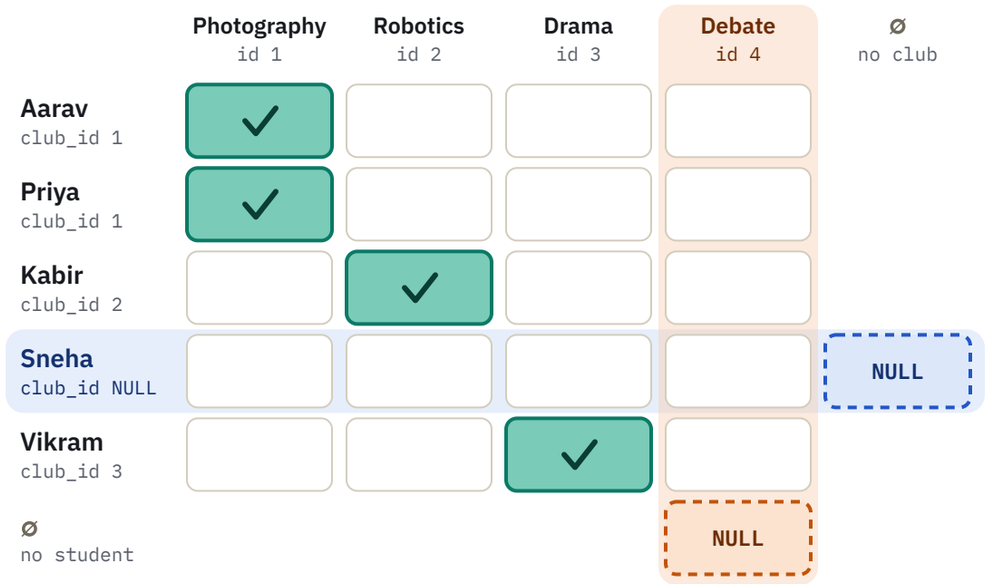
```
FROM students s FULL JOIN clubs c ON s.club_id = c.club_id
```

MySQL has no FULL JOIN: write LEFT JOIN **UNION** RIGHT JOIN.

Every join is a filtered grid.

Rows are students, columns are clubs, each cell is one possible pair. A join decides which cells to keep.

5 STUDENTS × 4 CLUBS



1 **CROSS JOIN = the whole grid**

No ON. Every student paired with every club: 5 × 4 = **20 rows**.

2 **ON club_id = club_id = light up cells**

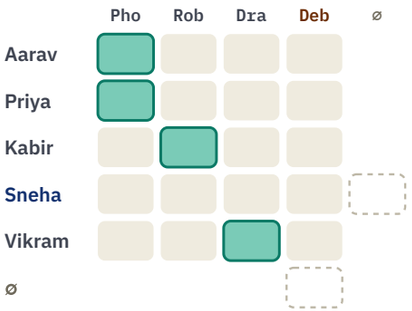
Only the cells where the ids agree stay lit. Those **4 cells are INNER JOIN**.

3 **Outer joins rescue empty lines**

Sneha's row and **Debate's column** have no lit cell. The dashed `NULL` cell pairs an orphan with nothing so it survives.

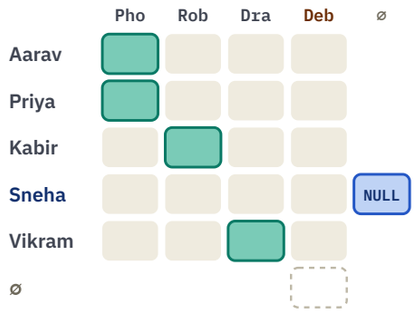
ids match · pair exists, no match · NULL rescue

INNER — 4 ROWS



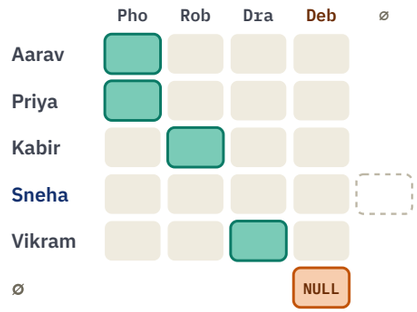
Lit cells only. Both orphans are gone.

LEFT — 5 ROWS



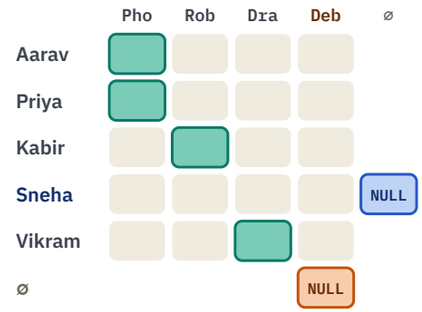
Lit cells + **Sneha**'s NULL cell.

RIGHT — 5 ROWS



Lit cells + **Debate**'s NULL cell.

FULL — 6 ROWS



Lit cells + both NULL cells.

Inner, outer, left, right, full.

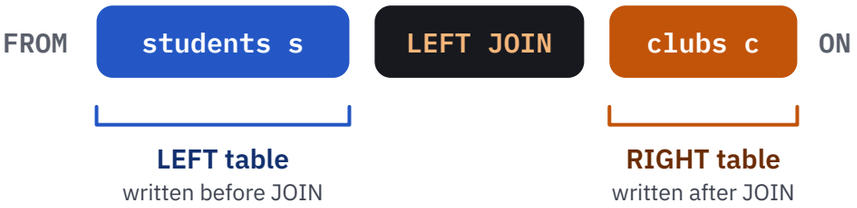
Five words, one picture: three zones, and each keyword keeps a different stretch of them.

THREE ZONES, FOUR KEYWORDS · EACH BAR IS WHAT IT KEEPS

INNER = middle only · **OUTER** = reaches a side · **LEFT / RIGHT** = which side · **FULL** = both



LEFT / RIGHT = WHICH TABLE YOU TYPED FIRST



LEFT JOIN protects the left table: **Sneha** survives.
RIGHT JOIN protects the right table: **Debate** survives.

WHICH WORDS ARE OPTIONAL?

required · optional

`INNER` `JOIN`

INNER can be dropped. Bare JOIN is INNER JOIN.

`LEFT | RIGHT | FULL` `OUTER` `JOIN`

OUTER can be dropped. LEFT, RIGHT or FULL can never be.

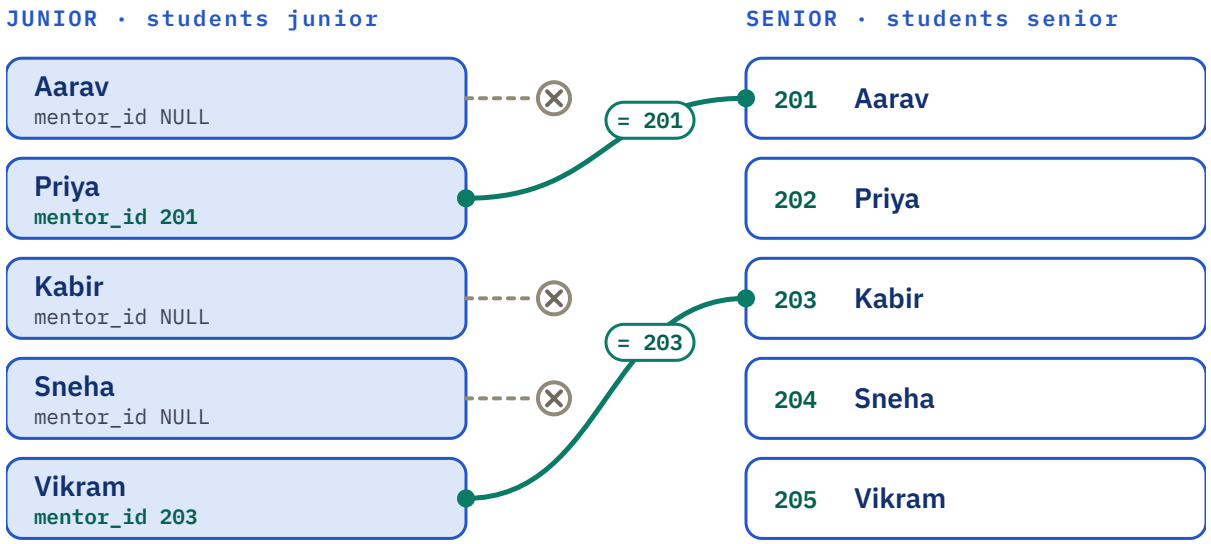
× ~~FULL INNER JOIN~~

Contradiction: INNER drops every orphan, FULL keeps every orphan.

Same table twice, and shortcuts.

Not new join types: a join where both sides are one table, and two ways to type less.

SELF JOIN · ONE TABLE, TWO HATS (ALIASES)



`ON junior.mentor_id = senior.student_id`

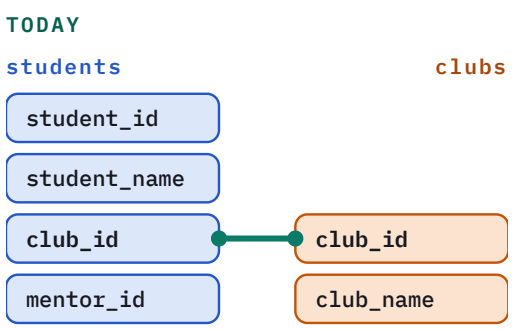
Who mentors whom? Mentor and mentee both live in **students**.

```
SELECT junior.student_name AS student,
       senior.student_name AS mentor
FROM students junior
LEFT JOIN students senior
  ON junior.mentor_id = senior.student_id;
```

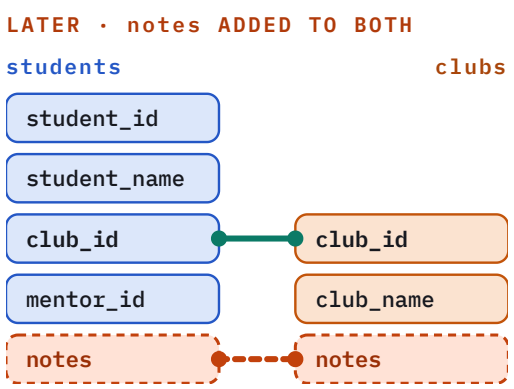
STUDENT	MENTOR
Aarav	NULL
Priya	**Aarav**
Kabir	NULL
Sneha	NULL
Vikram	**Kabir**

LEFT JOIN, not INNER, so students with no mentor still appear.

NATURAL JOIN · LINKS EVERY SAME-NAMED COLUMN



One shared name, one link: same as **`ON club_id`**. Result: Aarav, Priya, Kabir, Vikram.



A coincidental name becomes a second link. **Same SQL, different rows**: matches vanish unless notes also agree.

THREE WAYS TO WRITE THE SAME INNER JOIN

ON — explicit · default

```
JOIN clubs c ON s.club_id = c.club_id
```

USING — you name the column

```
JOIN clubs USING (club_id)
```

`s.club_id` + `c.club_id` → merged into one **`club_id`**

NATURAL — implicit · throwaway only

```
NATURAL JOIN clubs
```

Peek, don't fetch.

A JOIN returns every matching pair. EXISTS only asks whether a match is there, so a row can never repeat.

ONE ROW PER CLUB · THREE QUERIES

"Which clubs have members?" and "which have none?"

CLUBS where we start	MEMBERS students.club_id = club_id	INNER JOIN fetches every pair	WHERE EXISTS asks: any member?	WHERE NOT EXISTS asks: no member?
Photography	Aarav	Photography	Photography ✓	⊗ dropped
	Priya	Photography dup	2 members, still 1 row	
Robotics	Kabir	Robotics	Robotics ✓	⊗ dropped
Drama	Vikram	Drama	Drama ✓	⊗ dropped
Debate	*no members*	⊗ dropped	⊗ dropped	Debate ✓
OUTPUT SIZE		4 rows. Photography twice, so you need DISTINCT.	3 rows. One per club, no DISTINCT needed.	1 row. Only the empty club survives.

THE TWO PATTERNS

Semi-join: active clubs

```
SELECT c.club_name
FROM clubs c
WHERE EXISTS (
  SELECT 1 FROM students s
  WHERE s.club_id = c.club_id
);
```

Anti-join: empty clubs

```
SELECT c.club_name
FROM clubs c
WHERE NOT EXISTS (
  SELECT 1 FROM students s
  WHERE s.club_id = c.club_id
);
```

EXISTS = keep me if a match is found. **NOT EXISTS** = keep me if none is. Start from `students` instead to find the club-less: **Sneha**.

WHICH ONE DO I NEED?

Columns from **both** tables	JOIN
Only **yes/no**: does a match exist?	EXISTS
Find the **orphans**: no match at all	NOT EXISTS

Which join do I reach for?

ASK 1 Compare rows within one table?

no

ASK 2 Need every possible pair?

no

ASK 3 Need columns from both tables?

yes

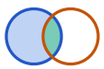
ASK 4 Which unmatched rows must survive?

ASK 1 → yes → **SELF JOIN** — one table, two aliases

ASK 2 → yes → **CROSS JOIN** — every pair, no ON (5 × 4 = 20)

ASK 3 → no → **EXISTS / NOT EXISTS** — match check, no duplicates

ASK 4 →

- none →  **INNER JOIN** — keeps matches only
- left table's →  **LEFT JOIN** — keeps the left table's orphans (Sneha)
- right table's →  **RIGHT JOIN** — keeps the right table's orphans (Debate)
- both →  **FULL JOIN** — keeps both sets of orphans

FOUR GOTCHAS

NULL never matches.
Sneha's club_id is NULL, so ON s.club_id = c.club_id can never pair her. That is why she is an orphan.

Bare JOIN = INNER JOIN.
OUTER is optional after LEFT, RIGHT or FULL. The side word never is. FULL INNER JOIN does not exist.

MySQL has no FULL JOIN.
Write LEFT JOIN ... UNION ... RIGHT JOIN. Use UNION, not UNION ALL, or every matched row appears twice.

Skip NATURAL JOIN in real code.
A new same-named column silently changes the join. Use ON, or USING (col). For RIGHT JOIN, swap the tables and use LEFT.